Exp No: 5	**Comparison of Three Popular CNN Architectures: VGG, ResNet and GoogLeNet for Image Classification**

AIM:

To implement and compare the performance of three popular CNN architectures, VGG, ResNet and GoogLeNet, for image classification.

ALGORITHM / PROCEDURE:

1. Load the labeled image dataset (8x8 grayscale handwritten digits, 10 classes), scale it to 0-1 and split it 80% / 20% into training and test sets.
2. **VGG:** blocks of stacked 3x3 convolutions with ReLU, each block followed by 2x2 max pooling, then fully connected layers.
3. **ResNet:** residual blocks that compute `F(x) + x` using a skip connection (a 1x1 convolution on the skip path when the number of channels changes), followed by global average pooling.
4. **GoogLeNet:** Inception blocks with parallel branches (1x1 convolution, 1x1 then 3x3 convolution, and 3x3 max pooling then 1x1 convolution) joined by channel concatenation, followed by global average pooling.
5. Implement every layer from scratch in NumPy with forward and backward passes. The gradients of all three networks were verified against numerical gradients.
6. Train all three networks with identical settings: cross-entropy loss, Adam (lr = 0.003), batch size 32, 20 epochs, same random seed.
7. Record training loss and test accuracy after every epoch, then compare parameters, accuracy, loss and training time.

Fig 1: VGG-style network

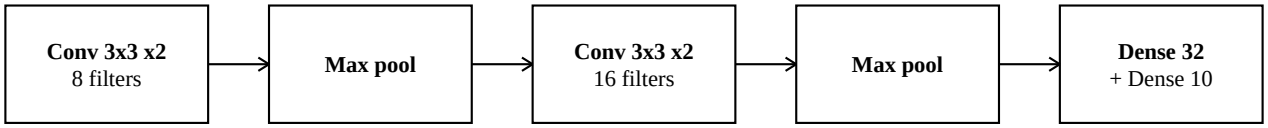
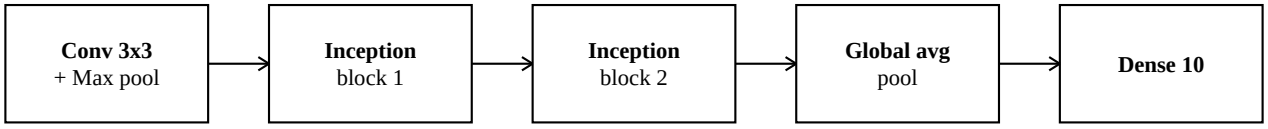


Fig 2: ResNet-style network



Fig 3: GoogLeNet-style network



Commands / Settings used:

Model	Main building block	Parameters
VGG	Stacked 3x3 convolutions + max pooling + dense layers	6562
ResNet	Residual (skip) blocks + global average pooling	5050
GoogLeNet	Inception blocks (1x1, 3x3, pooling) + global average pooling	3288

PROGRAM:

```
import time
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view as swv
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

# ---------- Data: labeled images (8x8 grayscale handwritten digits) ----------
d = load_digits()
X = d.data.reshape(-1, 8, 8, 1) / 16.0
y = d.target
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
Ytr = np.eye(10)[ytr]


# ---------- Helper functions for convolution (im2col / col2im) ----------
def im2col(x, k, pad_val=0.0):
    p = k // 2
    xp = np.pad(x, ((0, 0), (p, p), (p, p), (0, 0)), constant_values=pad_val)
    return swv(xp, (k, k), axis=(1, 2))              # (N, H, W, C, k, k)


def col2im(dw, k, shape):
    N, H, W, C = shape
    p = k // 2
    dxp = np.zeros((N, H + 2 * p, W + 2 * p, C))
    for i in range(k):
        for j in range(k):
            dxp[:, i:i + H, j:j + W, :] += dw[:, :, :, :, i, j]
    return dxp[:, p:p + H, p:p + W, :]


# ---------- Layers (each has forward / backward) ----------
class Conv:
    def __init__(self, cin, cout, k, rng):
        self.k = k
        self.W = rng.standard_normal((cin * k * k, cout)) * np.sqrt(2 / (cin * k * k))
        self.b = np.zeros(cout)
        self.gW, self.gb = np.zeros_like(self.W), np.zeros_like(self.b)

    def forward(self, x):
        self.shape = x.shape
        N, H, W, C = x.shape
        self.cols = im2col(x, self.k).reshape(N * H * W, -1)
        return (self.cols @ self.W + self.b).reshape(N, H, W, -1)

    def backward(self, g):
        N, H, W, C = self.shape
        g2 = g.reshape(N * H * W, -1)
        self.gW, self.gb = self.cols.T @ g2, g2.sum(0)
        dc = (g2 @ self.W.T).reshape(N, H, W, C, self.k, self.k)
        return col2im(dc, self.k, self.shape)

    def pairs(self):
        return [(self.W, self.gW), (self.b, self.gb)]
class Dense:
    def __init__(self, nin, nout, rng):
```

```
        self.W = rng.standard_normal((nin, nout)) * np.sqrt(2 / nin)
        self.b = np.zeros(nout)
        self.gW, self.gb = np.zeros_like(self.W), np.zeros_like(self.b)

    def forward(self, x):
        self.x = x
        return x @ self.W + self.b

    def backward(self, g):
        self.gW, self.gb = self.x.T @ g, g.sum(0)
        return g @ self.W.T

    def pairs(self):
        return [(self.W, self.gW), (self.b, self.gb)]


class ReLU:
    def forward(self, x):
        self.m = x > 0
        return x * self.m

    def backward(self, g):
        return g * self.m

    def pairs(self):
        return []


class MaxPool2:                                     # 2x2 max pooling, stride 2
    def forward(self, x):
        N, H, W, C = x.shape
        self.xr = x.reshape(N, H // 2, 2, W // 2, 2, C)
        self.out = self.xr.max(axis=(2, 4))
        return self.out

    def backward(self, g):
        m = self.xr == self.out[:, :, None, :, None, :]
        m = m / m.sum(axis=(2, 4), keepdims=True)
        N, h, _, w, _, C = self.xr.shape
        return (m * g[:, :, None, :, None, :]).reshape(N, 2 * h, 2 * w, C)

    def pairs(self):
        return []


class MaxPool3S1:                                   # 3x3 max pooling, stride 1
    def forward(self, x):
        self.shape = x.shape
        N, H, W, C = x.shape
        win = im2col(x, 3, -1e9).reshape(N, H, W, C, 9)
        self.idx = win.argmax(-1)
        return win.max(-1)

    def backward(self, g):
        N, H, W, C = self.shape
        oh = np.zeros((N, H, W, C, 9))
        np.put_along_axis(oh, self.idx[..., None], g[..., None], -1)
        return col2im(oh.reshape(N, H, W, C, 3, 3), 3, self.shape)

    def pairs(self):
        return []


class Flatten:
    def forward(self, x):
        self.shape = x.shape
        return x.reshape(len(x), -1)

    def backward(self, g):
        return g.reshape(self.shape)

    def pairs(self):
        return []
class GlobalAvgPool:
    def forward(self, x):
```

```
        self.shape = x.shape
        return x.mean(axis=(1, 2))

    def backward(self, g):
        N, H, W, C = self.shape
        return np.broadcast_to(g[:, None, None, :] / (H * W), self.shape).copy()

    def pairs(self):
        return []


class Seq:
    def __init__(self, layers):
        self.layers = layers

    def forward(self, x):
        for l in self.layers:
            x = l.forward(x)
        return x

    def backward(self, g):
        for l in reversed(self.layers):
            g = l.backward(g)
        return g

    def pairs(self):
        return [p for l in self.layers for p in l.pairs()]


class Residual:                                     # output = F(x) + shortcut(x)
    def __init__(self, main, shortcut=None):
        self.main, self.short = main, shortcut

    def forward(self, x):
        s = self.short.forward(x) if self.short else x
        return self.main.forward(x) + s

    def backward(self, g):
        ds = self.short.backward(g) if self.short else g
        return self.main.backward(g) + ds

    def pairs(self):
        return self.main.pairs() + (self.short.pairs() if self.short else [])


class Inception:                                    # parallel branches, concatenated
    def __init__(self, branches):
        self.branches = branches

    def forward(self, x):
        outs = [b.forward(x) for b in self.branches]
        self.sizes = np.cumsum([o.shape[-1] for o in outs])[:-1]
        return np.concatenate(outs, axis=-1)

    def backward(self, g):
        parts = np.split(g, self.sizes, axis=-1)
        return sum(b.backward(p) for b, p in zip(self.branches, parts))

    def pairs(self):
        return [p for b in self.branches for p in b.pairs()]


# ---------- The three architectures (scaled-down versions) ----------
def vgg(rng):
    return Seq([Conv(1, 8, 3, rng), ReLU(), Conv(8, 8, 3, rng), ReLU(), MaxPool2(),
                Conv(8, 16, 3, rng), ReLU(), Conv(16, 16, 3, rng), ReLU(), MaxPool2(),
                Flatten(), Dense(64, 32, rng), ReLU(), Dense(32, 10, rng)])


def resnet(rng):
    block1 = Residual(Seq([Conv(8, 8, 3, rng), ReLU(), Conv(8, 8, 3, rng)]))
    block2 = Residual(Seq([Conv(8, 16, 3, rng), ReLU(), Conv(16, 16, 3, rng)]),
                      Seq([Conv(8, 16, 1, rng)]))     # 1x1 projection shortcut
    return Seq([Conv(1, 8, 3, rng), ReLU(), block1, ReLU(), MaxPool2(),
                block2, ReLU(), GlobalAvgPool(), Dense(16, 10, rng)])
```

```
def googlenet(rng):
    def inception(cin, c1, c3r, c3, cp):
        return Inception([
            Seq([Conv(cin, c1, 1, rng), ReLU()]),
            Seq([Conv(cin, c3r, 1, rng), ReLU(), Conv(c3r, c3, 3, rng), ReLU()]),
            Seq([MaxPool3S1(), Conv(cin, cp, 1, rng), ReLU()])])
    return Seq([Conv(1, 8, 3, rng), ReLU(), MaxPool2(),
                inception(8, 8, 6, 12, 8),            # output channels: 28
                inception(28, 12, 8, 16, 8),          # output channels: 36
                GlobalAvgPool(), Dense(36, 10, rng)])


# ---------- Training utilities ----------
def softmax(z):
    e = np.exp(z - z.max(1, keepdims=True))
    return e / e.sum(1, keepdims=True)


def predict(model, X):
    return np.vstack([softmax(model.forward(X[i:i + 256])) for i in range(0, len(X), 256)])


def train(name, build, epochs=20, batch=32, lr=0.003):
    rng = np.random.default_rng(0)
    model = build(rng)
    n_params = sum(a.size for a, _ in model.pairs())
    m = v = None
    t = 0
    hist = {"loss": [], "acc": []}
    t0 = time.time()
    order_rng = np.random.default_rng(1)
    for ep in range(1, epochs + 1):
        idx = order_rng.permutation(len(Xtr))
        run = 0.0
        for s in range(0, len(idx), batch):
            b = idx[s:s + batch]
            P = softmax(model.forward(Xtr[b]))
            run += -np.sum(Ytr[b] * np.log(P + 1e-12))
            model.backward((P - Ytr[b]) / len(b))
            pr = model.pairs()
            if m is None:
                m = [np.zeros_like(a) for a, _ in pr]
                v = [np.zeros_like(a) for a, _ in pr]
            t += 1
            for i, (a, g) in enumerate(pr):             # Adam update
                m[i] = 0.9 * m[i] + 0.1 * g
                v[i] = 0.999 * v[i] + 0.001 * g * g
                a -= lr * (m[i] / (1 - 0.9**t)) / (np.sqrt(v[i] / (1 - 0.999**t)) + 1e-8)
        hist["loss"].append(run / len(Xtr))
        hist["acc"].append(np.mean(predict(model, Xte).argmax(1) == yte) * 100)
    secs = time.time() - t0
    Ptr, Pte = predict(model, Xtr), predict(model, Xte)
    res = dict(params=n_params, secs=secs,
               train_acc=np.mean(Ptr.argmax(1) == ytr) * 100,
               test_acc=np.mean(Pte.argmax(1) == yte) * 100,
               test_loss=-np.mean(np.log(Pte[np.arange(len(yte)), yte] + 1e-12)))
    return hist, res


if __name__ == "__main__":
    models = {"VGG": vgg, "ResNet": resnet, "GoogLeNet": googlenet}
    H, R = {}, {}
    for name, fn in models.items():
        print(f"Training {name} ...")
        H[name], R[name] = train(name, fn)

    print(f"\n{'Epoch':<7}" + "".join(f"{n + ' loss':<15}{n + ' acc':<14}" for n in models))
    for e in [1, 5, 10, 15, 20]:
        print(f"{e:<7}" + "".join(
            f"{H[n]['loss'][e-1]:<15.4f}{H[n]['acc'][e-1]:<14.2f}" for n in models))

    print(f"\n{'Model':<11}{'Params':<8}{'Train acc':<11}{'Test acc':<10}"
          f"{'Test loss':<11}{'Time (s)'}")
    print("-" * 58)
    for n, r in R.items():
```

```
    print(f"{n:<11}{r['params']:<8}{r['train_acc']:<11.2f}{r['test_acc']:<10.2f}"
          f"{r['test_loss']:<11.4f}{r['secs']:.1f}")
best = max(R, key=lambda n: (R[n]["test_acc"], -R[n]["test_loss"]))
print(f"\nBest test accuracy: {best} ({R[best]['test_acc']:.2f}%)")

fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
for n in models:
    ax[0].plot(range(1, 21), H[n]["loss"], label=n)
    ax[1].plot(range(1, 21), H[n]["acc"], label=n)
ax[0].set(xlabel="Epoch", ylabel="Training loss", title="Loss vs epoch", yscale="log")
ax[1].set(xlabel="Epoch", ylabel="Test accuracy (%)", title="Test accuracy vs epoch")
for a in ax:
    a.grid(alpha=.3)
    a.legend()
plt.tight_layout()
plt.savefig("img/exp5_curves.png", dpi=200)
```

OUTPUT:

Terminal - VGG vs ResNet vs GoogLeNet

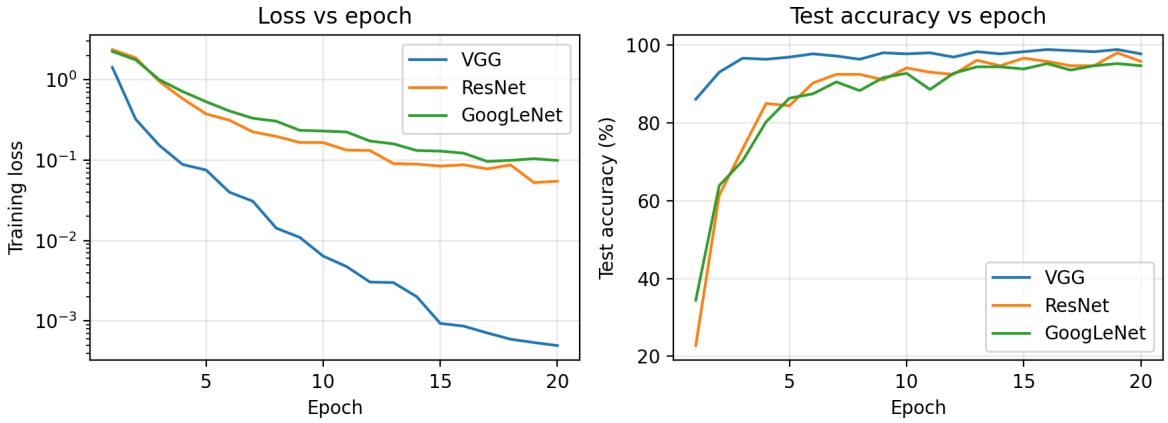
```
root@ip-10-10-5-10:~# python3 exp5.py
Training VGG ...
Training ResNet ...
Training GoogLeNet ...

Epoch  VGG loss       VGG acc       ResNet loss    ResNet acc    GoogLeNet loss GoogLeNet acc
1      1.4230         86.11         2.3678         22.78         2.2433         34.44
5      0.0755         96.94         0.3756         84.44         0.5325         86.39
10     0.0064         97.78         0.1652         94.17         0.2301         92.78
15     0.0009         98.33         0.0844         96.67         0.1294         93.89
20     0.0005         97.78         0.0547         95.83         0.0991         94.72

Model      Params  Train acc  Test acc  Test loss  Time (s)
-----------------------------------------------------------
VGG        6562    100.00     97.78     0.0404     3.4
ResNet     5050    98.40      95.83     0.1569     4.9
GoogLeNet  3288    96.94      94.72     0.1888     5.5

Best test accuracy: VGG (97.78%)
```

Fig 4: Training loss and test accuracy of the three networks



RESULT:

Thus VGG, ResNet and GoogLeNet style networks were implemented from scratch and compared under identical training settings. The VGG network gave the best result with 97.78% test accuracy (test loss 0.0404), followed by ResNet with 95.83% and GoogLeNet with 94.72%. VGG also reduced its training loss fastest and has the most parameters (6562). These are very small, shallow versions trained without batch normalization on 8x8 images, so this ranking applies to this setup only and does not show the benefit of residual or Inception blocks in very deep networks.